Analytics Engineering: Session 12

Practice Session III: Production-Ready Data Quality

Goal

Consolidate Sessions 9-11: Build a production-ready project with comprehensive testing, governance, incremental models, and snapshots.

Tasks:

1. Implement a full test suite across all layers
2. Add contracts and governance to mart models
3. Convert a model to incremental
4. Create a snapshot
5. Use `dbt build` with selectors

Exercise 1: Full Test Suite

Implement tests across every layer:

Staging:

```yaml
- name: stg_orders
  columns:
    - name: order_id
      tests: [unique, not_null]
    - name: customer_id
      tests: [not_null]
```

Marts:

```yaml
- name: mart_orders
  columns:
    - name: order_id
      tests: [unique, not_null]
```

Exercise 1: Custom Generic Tests

Create a reusable test macro and apply it:

```
-- macros/test_is_positive.sql
{% test is_positive(model, column_name) %}
    select *
    from {{ model }}
    where {{ column_name }} < 0
{% endtest %}
```

```
# Apply to mart models
- name: total_amount
  tests:
    - is_positive
```

Exercise 2: Add Contracts to Marts

Enforce schema on all mart models:

```yaml
models:
  - name: dim_customers
    config:
      contract: {enforced: true}
    columns:
      - name: customer_id
        data_type: int
        constraints:
          - type: not_null
          - type: unique
      - name: email
        data_type: varchar
      - name: customer_segment
        data_type: varchar
```

Exercise 3: Incremental Model

Convert `mart_orders` to an incremental model:

```
{{ config(
    materialized='incremental',
    unique_key='order_id'
) }}

select *
from {{ ref('stg_orders') }}
left join {{ ref('int_order_items_summary') }} using (order_id)
left join {{ ref('int_order_shipping') }} using (order_id)
{% if is_incremental() %}
  where order_date > (select max(order_date) from {{ this }})
{% endif %}
```

Test with: `dbt run -s mart_orders --full-refresh`

Exercise 4: Create a Snapshot

Track customer changes over time:

```
{% snapshot customer_snapshot %}
{{
  config(
    target_schema='snapshots',
    unique_key='customer_id',
    strategy='timestamp',
    updated_at='updated_at'
  )
}}
select * from {{ source('raw', 'customers') }}
{% endsnapshot %}
```

Run with: `dbt snapshot`

Exercise 5: `dbt build` with Selectors

Practice different selector patterns:

```
# Build everything
dbt build

# Build a specific model and its tests
dbt build -s mart_orders

# Build a model and everything downstream
dbt build -s mart_orders+

# Build everything upstream of a model (inclusive)
dbt build -s +mart_orders

# Build only staging layer
dbt build -s staging

# Build only models with a tag
```

Exercise 6: Version a Critical Model

Version `dim_customers` and simulate a migration:

1. Create v2 with a new column (`loyalty_tier`)
2. Deprecate v1 with a date
3. Update downstream models to reference v2
4. Run `dbt build` and observe the deprecation warnings

Checklist

- [ ] Generic tests on all primary keys and foreign keys
- [ ] At least one singular test for a business rule
- [ ] At least one custom generic test
- [ ] Contracts on all mart models
- [ ] One incremental model with full-refresh tested
- [ ] One snapshot with `dbt snapshot` verified
- [ ] `dbt build` passes with zero failures

What have we learned in this session

- Implemented a comprehensive test suite (generic, singular, custom generic)
- Added contracts to enforce schema on mart models
- Converted a model to incremental materialization
- Created a snapshot for historical tracking
- Versioned a model and practiced migration
- Used `dbt build` with different selectors

Next Session: State, Debugging, Deployment & CI/CD.